

Here's the **Count-the-1's (stream of bytes)** test explained **step-by-step** in the easiest way, specifically for a file of **25600 bytes** named output.dat.

---

### What you have

- A binary file output.dat with **25600 bytes**
  - Each byte = **8 bits**
- 

### Step 1: Convert each byte into a "letter" A–E

For every byte, count how many **1 bits** it has (0 to 8). Call that number  $w$ .

Then map  $w$  to a letter:

- **A** if  $w = 0,1,2$
- **B** if  $w = 3$
- **C** if  $w = 4$
- **D** if  $w = 5$
- **E** if  $w = 6,7,8$

So your 25600 bytes become **25600 letters**.

### Why these probabilities matter

If bytes are truly random, then number of 1s in a byte is Binomial(8, 1/2), so:

- $p(A) = 37/256$
- $p(B) = 56/256$
- $p(C) = 70/256$
- $p(D) = 56/256$
- $p(E) = 37/256$

(These come from (1,8,28,56,70,56,28,8,1)/256 grouped into A–E.)

---

### Step 2: Build overlapping 5-letter "words"

From the letter stream  $L_1, L_2, \dots, L_{25600}$ , make words of length 5:

- Word 1:  $L_1 L_2 L_3 L_4 L_5$
- Word 2:  $L_2 L_3 L_4 L_5 L_6$
- ...
- Last word:  $L_{25596} L_{25597} L_{25598} L_{25599} L_{25600}$

So the number of overlapping 5-letter words is:

$$N = 25600 - 4 = 25596$$

There are  $5^5 = 3125$  possible 5-letter words.

Count how many times each of the 3125 words occurs:

- call those observed counts  $O_i^{(5)}, i = 1..3125$
- 

### Step 3: Also build overlapping 4-letter “words” (for Q4)

Diehard also needs 4-letter word counts.

For each 5-letter word  $L_i L_{i+1} L_{i+2} L_{i+3} L_{i+4}$ , take its **last 4 letters**:

$$(L_{i+1} L_{i+2} L_{i+3} L_{i+4})$$

That gives you exactly **N = 25596** overlapping 4-letter words.

There are  $5^4 = 625$  possible 4-letter words.

Count how many times each of the 625 words occurs:

- call those observed counts  $O_j^{(4)}, j = 1..625$
- 

### Step 4: Compute the expected counts

#### Expected count for a specific 5-letter word

Suppose a 5-letter word is  $ABCDE$ . Its probability under randomness is:

$$P(ABCDE) = p(A)p(B)p(C)p(D)p(E)$$

So expected count is:

$$E^{(5)} = N \cdot p(A)p(B)p(C)p(D)p(E)$$

#### Expected count for a specific 4-letter word

Similarly for 4 letters  $BCDE$ :

$$E^{(4)} = N \cdot p(B)p(C)p(D)p(E)$$

---

### Step 5: Compute Q5 (chi-square for 5-letter words)

$$Q5 = \sum_{i=1}^{3125} \frac{(O_i^{(5)} - E_i^{(5)})^2}{E_i^{(5)}}$$


---

**Step 6: Compute Q4 (chi-square for 4-letter words)**

$$Q4 = \sum_{j=1}^{625} \frac{(O_j^{(4)} - E_j^{(4)})^2}{E_j^{(4)}}$$


---

**Step 7: Final Diehard statistic**

Diehard uses:

$$S = Q5 - Q4$$

And then computes a z-score:

$$z = \frac{S - 2500}{\sqrt{5000}}$$

and p-value:

$$p = 1 - \Phi(z)$$


---

**Very important warning for your file size (25600 bytes)**

For **Diehard**, the classic sample is  $N = 256000$  words.

You have only:

$$N = 25596$$

Average expected count per 5-letter cell:

$$\frac{N}{3125} \approx \frac{25596}{3125} \approx 8.19$$

That's **small**. Many specific words will have expected counts even smaller (because probabilities differ), so chi-square can become **noisy**.

So with 25600 bytes:

- you can compute  $Q_4$ ,  $Q_5$ , and  $Q_5 - Q_4$ ,
- but results will be **less stable** than the original Diehard test.

If you want a more reliable run, use **256004 bytes** (classic) so  $N = 256000$ .

---

### What values should you “expect” to see?

- The Diehard combined stat  $S = Q_5 - Q_4$  is expected around:
  - mean  $\approx$  **2500**
  - std  $\approx$  **70.71**  
but with your smaller  $N$ , you may see larger fluctuations.

---

If you want, I can also give you a **single complete C program** that reads output.dat (25600 bytes), prints the letter frequencies (A–E), then prints  $Q_4$ ,  $Q_5$ ,  $Q_5 - Q_4$ ,  $z$  and  $p$ -value — so you can directly run it and see the output.

/\*

Diehard "Count-the-1's" test on a STREAM OF BYTES

-----

You said: output.dat already exists and has 256000 bytes.

We will:

1) Read exactly 256000 bytes from output.dat (binary)

2) Map each byte  $\rightarrow$  letter in {A,B,C,D,E} using:

A: 0-2 ones, B:3, C:4, D:5, E:6-8

3) Form overlapping 5-letter words:

$N = n\_bytes - 4 = 255996$

Count all 3125 possible 5-letter words (f5)

Also count suffix 4-letter words (625 possible) (f4)

4) Compute:

$Q_4 = \text{sum}_{\{625\}} (O-E)^2/E$

$Q_5 = \text{sum}_{\{3125\}} (O-E)^2/E$

stat =  $Q_5 - Q_4$

And print  $z$ -score &  $p$ -value using Diehard's classic approximation:

mean=2500, std=sqrt(5000),  $p = 1 - \Phi(z)$

Build:

```
gcc -O2 -std=c11 cnt1s_stream.c -lm -o cnt1s
```

Run:

```
./cnt1s
```

```
*/
```

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
static inline int popcount8(uint8_t x) {
```

```
    int c = 0;
```

```
    for (int i = 0; i < 8; ++i) c += (x >> i) & 1u;
```

```
    return c;
```

```
}
```

```
// Map byte -> digit 0..4 representing A..E
```

```
// A:0-2 ones, B:3, C:4, D:5, E:6-8
```

```
static inline uint8_t byte_to_letter(uint8_t b) {
```

```
    int w = popcount8(b);
```

```
    if (w <= 2) return 0; // A
```

```
    if (w == 3) return 1; // B
```

```
    if (w == 4) return 2; // C
```

```
    if (w == 5) return 3; // D
```

```
    return 4; // E
```

```
}
```

```

static inline double Phi(double z) {
    return 0.5 * (1.0 + erf(z / M_SQRT2));
}

// Expected count for index idx representing an L-letter base-5 word
static double expected_count(int idx, int L, double N, const double prob[5]) {
    double E = N;
    for (int j = 0; j < L; ++j) {
        int digit = idx % 5;
        E *= prob[digit];
        idx /= 5;
    }
    return E;
}

// Encode 5 digits (base-5) into [0..3124]
static inline int code5(uint8_t a, uint8_t b, uint8_t c, uint8_t d, uint8_t e) {
    return (((((int)a) * 5 + (int)b) * 5 + (int)c) * 5 + (int)d) * 5 + (int)e);
}

int main(void) {
    const char *fn = "output.dat";

    // You said file has 256000 bytes
    const size_t n_bytes = 256000;
    const size_t N = n_bytes - 4; // overlapping 5-letter words

    // Letter probabilities under true randomness for this mapping
    const double prob[5] = {
        37.0 / 256.0, // A
        56.0 / 256.0, // B

```

```

70.0 / 256.0, // C
56.0 / 256.0, // D
37.0 / 256.0 // E
};

// Diehard's classic normal approximation for (Q5 - Q4)
const double mean = 2500.0;
const double stdv = sqrt(5000.0);

// ---- Read file ----
FILE *fp = fopen(fn, "rb");
if (!fp) {
    perror("fopen output.dat");
    return 1;
}

uint8_t *buf = (uint8_t*)malloc(n_bytes);
if (!buf) {
    fprintf(stderr, "Out of memory allocating %zu bytes\n", n_bytes);
    fclose(fp);
    return 1;
}

size_t got = fread(buf, 1, n_bytes, fp);
fclose(fp);

if (got != n_bytes) {
    fprintf(stderr, "File size mismatch: expected %zu bytes, got %zu bytes\n", n_bytes, got);
    free(buf);
    return 1;
}

```

```

// ---- Convert bytes -> letters ----
uint8_t *L = (uint8_t*)malloc(n_bytes);
if (!L) {
    fprintf(stderr, "Out of memory allocating letters\n");
    free(buf);
    return 1;
}

// Optional: letter frequency check
uint64_t letter_cnt[5] = {0,0,0,0,0};

for (size_t i = 0; i < n_bytes; ++i) {
    uint8_t d = byte_to_letter(buf[i]);
    L[i] = d;
    ++letter_cnt[d];
}

// ---- Count overlapping 5-letter words + suffix 4-letter words ----
uint32_t f5[3125] = {0};
uint32_t f4[625] = {0};

for (size_t i = 0; i < N; ++i) {
    int w5 = code5(L[i], L[i+1], L[i+2], L[i+3], L[i+4]); // 0..3124
    ++f5[w5];

    int suffix4 = w5 % 625; // last 4 letters
    ++f4[suffix4];
}

// ---- Compute Q4, Q5 ----

```



```
double Q4 = 0.0, Q5 = 0.0;
```

```
for (int i = 0; i < 625; ++i) {  
    double E = expected_count(i, 4, (double)N, prob);  
    double diff = (double)f4[i] - E;  
    Q4 += (diff * diff) / E;  
}
```

```
for (int i = 0; i < 3125; ++i) {  
    double E = expected_count(i, 5, (double)N, prob);  
    double diff = (double)f5[i] - E;  
    Q5 += (diff * diff) / E;  
}
```

```
double stat = Q5 - Q4;  
double z = (stat - mean) / stdv;  
double p = 1.0 - Phi(z);
```

```
// ---- Print results ----
```

```
printf("COUNT-THE-1's TEST (stream of bytes)\n");  
printf("File: %s\n", fn);  
printf("Bytes read: %zu\n", n_bytes);  
printf("Overlapping 5-letter words N = %zu\n\n", N);
```

```
printf("Letter frequencies (A..E):\n");  
printf(" A: %llu\n", (unsigned long long)letter_cnt[0]);  
printf(" B: %llu\n", (unsigned long long)letter_cnt[1]);  
printf(" C: %llu\n", (unsigned long long)letter_cnt[2]);  
printf(" D: %llu\n", (unsigned long long)letter_cnt[3]);  
printf(" E: %llu\n\n", (unsigned long long)letter_cnt[4]);
```

```
printf("Q4    = %.6f\n", Q4);  
printf("Q5    = %.6f\n", Q5);  
printf("Q5 - Q4 = %.6f\n", stat);  
printf("z-score = % .6f\n", z);  
printf("p-value = %.10f\n", p);
```

```
free(L);  
free(buf);  
return 0;  
}
```